10 CAHIERS DES CHARGES

Parcours progressif de développement web | du premier programme au projet full-stack

Public cible : étudiant en première année d'informatique. Objectif : réaliser 10 projets pédagogiques, déployer chaque réalisation gratuitement et construire un portfolio GitHub vérifiable.

Édition du 23 octobre 2026 | Périmètre : démonstration personnelle, sans usage commercial ni paiement réel.

Principe de progression

- Projets 1 à 6 : HTML, CSS, JavaScript, sans serveur à gérer.
- Projets 7 à 10 : application front-end et services Supabase ; l'étudiant apprend à manipuler des données persistantes et à sécuriser les accès.
- Chaque projet exige : dépôt GitHub public (sans secrets), README, captures d'écran, démonstration en ligne et tests documentés.
- Les durées estimées supposent que les notions sont apprises en parallèle ; la qualité prime sur le calendrier.

Répartition conseillée des hébergements

- GitHub Pages : pages statiques HTML/CSS/JavaScript (projets 1 à 6).
- Vercel Hobby : interfaces statiques et React de démonstration personnelle (projets 1 à 10). Attention : Hobby est réservé aux usages personnels non commerciaux.
- Supabase Free : authentification et PostgreSQL pour les projets 7 à 10. Le projet gratuit peut être suspendu après une période de faible activité ; le nombre de projets gratuits actifs est limité.
- Firebase Spark : alternative gratuite pour des prototypes front-end avec Auth/Firestore/Hosting sous quotas, mais ce dossier garde Supabase comme fil conducteur.
- Ne pas supposer que GitHub Pages héberge directement un backend Python, PHP ou Laravel : il sert les fichiers statiques.

Barème unique, sur 20 points

- Fonctionnalités obligatoires : 8 points.
- Fiabilité, validation et tests : 4 points.
- Qualité et lisibilité du code : 3 points.
- Design responsive et ergonomie : 2 points.
- README, GitHub et déploiement fonctionnel : 3 points.

Condition de passage au projet suivant

Au moins 14/20, toutes les exigences critiques vérifiées, et un lien de démonstration ouvrable sur un autre appareil. Les bonus ne compensent pas une faille de sécurité ou une fonction obligatoire absente.

PROJET 01 — Calculatrice interactive

Niveau	1 / Initiation	Durée indicative	2 à 3 jours
Technologies	HTML + CSS + JavaScript	Déploiement	GitHub Pages (priorité) ou Vercel

1. Objectif du projet

Créer une calculatrice utilisable au clavier et à la souris, avec un résultat fiable pour des calculs simples.

2. Utilisateur cible et contexte

Une personne voulant effectuer rapidement des calculs depuis son navigateur.

3. Fonctionnalités obligatoires

- Afficher un écran de saisie et un clavier de chiffres de 0 à 9.
- Gérer addition, soustraction, multiplication et division, ainsi que les décimales.
- Ajouter Effacer tout, Effacer le dernier caractère et Égal.
- Empêcher les saisies incohérentes (deux opérateurs successifs, plusieurs virgules dans un nombre).
- Afficher un message clair en cas de division par zéro et permettre une nouvelle opération.

4. Pages / écrans attendus

- Une page unique avec écran, grille de touches et courte aide.

5. Données et structure

État temporaire : saisie courante, premier opérande, opérateur et résultat. Aucune base de données.

6. Règles métier et contraintes

- Les calculs doivent respecter un comportement documenté : soit évaluation opération par opération, soit priorité mathématique expliquée.
- Ne pas utiliser eval() pour exécuter la saisie brute.
- Une opération incorrecte ne doit pas bloquer l’interface.

7. Plan de réalisation conseillé

- Étape 1 : Construire l’interface HTML et son style CSS.
- Étape 2 : Gérer les clics, puis les opérations et cas limites.
- Étape 3 : Ajouter le clavier et les tests manuels.
- Étape 4 : Publier le dossier sur GitHub Pages.

8. Recette : tests d’acceptation

- 12 + 7 = 19 ; 15 / 3 = 5 ; 7 / 0 produit une erreur lisible.
- Une suite d’appuis rapides ne provoque pas d’affichage cassé.
- L’interface tient sur un smartphone.

9. Bonus facultatifs

- Historique local des 10 dernières opérations.
- Thème clair/sombre.

10. Livrables et critères de validation

- Dépôt GitHub nommé clairement, avec commits significatifs et fichier README (objectif, installation, utilisation, captures, limites).
- Lien HTTPS du site réellement accessible ; aucune clé secrète publiée dans le dépôt.
- Minimum trois captures d’écran : interface principale, fonctionnalité clé, affichage mobile.
- Petite démonstration de 2 à 5 minutes et checklist de tests signée par le mentor.

PROJET 02 — Convertisseur d’unités

Niveau	2 / Initiation	Durée indicative	2 à 4 jours
Technologies	HTML + CSS + JavaScript	Déploiement	GitHub Pages ou Vercel

1. Objectif du projet

Offrir un formulaire intuitif pour convertir correctement des mesures sans recourir à un service externe.

2. Utilisateur cible et contexte

Élève, étudiant ou particulier convertissant ses valeurs courantes.

3. Fonctionnalités obligatoires

- Choisir une catégorie : température, distance et masse.
- Choisir unité de départ et unité d’arrivée.
- Saisir un nombre positif, négatif ou décimal lorsque c’est pertinent.
- Afficher instantanément le résultat avec l’unité correcte.
- Permettre d’inverser les unités de départ et d’arrivée.
- Afficher une erreur compréhensible lorsque l’entrée est vide ou non numérique.

4. Pages / écrans attendus

- Une page avec sélecteurs, champ valeur, résultat et formule expliquée.

5. Données et structure

Tables de conversion codées en JavaScript ; aucune base de données.

6. Règles métier et contraintes

- Prévoir °C / °F / K ; m / km / cm ; g / kg / lb.
- La température suit des formules spécifiques, et non un simple multiplicateur.
- Arrondir l’affichage sans fausser les calculs intermédiaires.

7. Plan de réalisation conseillé

- Étape 1 : Lister les unités et leurs formules.
- Étape 2 : Construire le formulaire.
- Étape 3 : Coder les fonctions de conversion.
- Étape 4 : Tester les changements de catégorie et le bouton Inverser.
- Étape 5 : Déployer le site statique.

8. Recette : tests d’acceptation

- 0 °C = 32 °F ; 1 km = 1 000 m ; 1 kg ≈ 2,20462 lb.
- Inverser deux unités conserve la cohérence du résultat.
- Changer la catégorie réinitialise les unités incompatibles.

9. Bonus facultatifs

- Conversion de volume.
- Historique local des conversions.

10. Livrables et critères de validation

- Dépôt GitHub nommé clairement, avec commits significatifs et fichier README (objectif, installation, utilisation, captures, limites).
- Lien HTTPS du site réellement accessible ; aucune clé secrète publiée dans le dépôt.
- Minimum trois captures d’écran : interface principale, fonctionnalité clé, affichage mobile.
- Petite démonstration de 2 à 5 minutes et checklist de tests signée par le mentor.

PROJET 03 — Jeu de devinette

Niveau	3 / Logique	Durée indicative	3 à 5 jours
Technologies	HTML + CSS + JavaScript	Déploiement	GitHub Pages ou Vercel

1. Objectif du projet

Construire un petit jeu complet avec une boucle de jeu, un score et une véritable gestion d'état.

2. Utilisateur cible et contexte

Tout visiteur souhaitant une partie rapide.

3. Fonctionnalités obligatoires

- Générer un entier aléatoire entre 1 et 100 au lancement.
- Laisser le joueur proposer un nombre.
- Indiquer « trop haut » ou « trop bas » après chaque essai.
- Limiter la partie à 7 essais et afficher les essais restants.
- Terminer clairement en cas de victoire ou défaite.
- Permettre de rejouer avec un nouveau nombre.

4. Pages / écrans attendus

- Page de jeu avec règles, saisie, historique des tentatives et bouton Rejouer.

5. Données et structure

Nombre secret, historique d'essais, état de partie et meilleur score conservé en localStorage.

6. Règles métier et contraintes

- Refuser les valeurs hors de 1 à 100, non entières ou vides sans consommer d'essai.
- Une tentative après la fin de partie ne doit rien modifier.
- Le nombre secret ne doit jamais être affiché avant la fin de la partie.

7. Plan de réalisation conseillé

- Étape 1 : Dessiner les états de la partie (initial, actif, gagné, perdu).
- Étape 2 : Programmer génération, validation et comparaisons.
- Étape 3 : Ajouter l'historique et le meilleur score.
- Étape 4 : Rendre la page responsive et publier.

8. Recette : tests d'acceptation

- 1, 100 et 50 sont acceptés ; 0, 101 et abc sont refusés.
- La 7e erreur entraîne une défaite.
- Rejouer réinitialise tous les compteurs.

9. Bonus facultatifs

- Modes facile et difficile.

- Tableau de scores local.

10. Livrables et critères de validation

- Dépôt GitHub nommé clairement, avec commits significatifs et fichier README (objectif, installation, utilisation, captures, limites).
- Lien HTTPS du site réellement accessible ; aucune clé secrète publiée dans le dépôt.
- Minimum trois captures d’écran : interface principale, fonctionnalité clé, affichage mobile.
- Petite démonstration de 2 à 5 minutes et checklist de tests signée par le mentor.

PROJET 04 — Gestionnaire de tâches

Niveau	4 / JavaScript pratique	Durée indicative	4 à 7 jours
Technologies	HTML + CSS + JavaScript + localStorage	Déploiement	GitHub Pages ou Vercel

1. Objectif du projet

Permettre à un utilisateur de gérer ses tâches quotidiennes et de retrouver ses données après un rechargement.

2. Utilisateur cible et contexte

Étudiant organisant ses devoirs et priorités.

3. Fonctionnalités obligatoires

- Créer une tâche avec titre obligatoire, description facultative, priorité et date limite.
- Afficher la liste des tâches et leur état (à faire / terminée).
- Modifier et supprimer une tâche avec confirmation adaptée.
- Filtrer les tâches : toutes, en cours, terminées.
- Trier par priorité ou date limite.
- Afficher le nombre de tâches restant à accomplir.
- Sauvegarder automatiquement les changements en localStorage.

4. Pages / écrans attendus

- Vue principale : formulaire, filtres, liste de tâches et compteur.

5. Données et structure

Chaque tâche possède un id unique, titre, description, priorité, date limite, statut et date de création. Tableau JSON conservé côté navigateur.

6. Règles métier et contraintes

- Un titre vide ou constitué uniquement d’espaces est interdit.
- Une tâche conserve le même identifiant après modification.
- Les données localStorage appartiennent au navigateur utilisé : elles ne se synchronisent pas entre appareils.

7. Plan de réalisation conseillé

- Étape 1 : Définir l’objet tâche.
- Étape 2 : Créer, afficher puis éditer et supprimer.
- Étape 3 : Ajouter filtres, tri et compteur.
- Étape 4 : Gérer localStorage et réouverture.
- Étape 5 : Tester puis publier.

8. Recette : tests d’acceptation

- Créer trois tâches, en terminer une, puis constater deux restantes.

- Rafraîchir la page conserve toutes les tâches.
- Supprimer une tâche ne modifie pas les autres.

9. Bonus facultatifs

- Recherche par mot-clé.
- Export/import JSON.

10. Livrables et critères de validation

- Dépôt GitHub nommé clairement, avec commits significatifs et fichier README (objectif, installation, utilisation, captures, limites).
- Lien HTTPS du site réellement accessible ; aucune clé secrète publiée dans le dépôt.
- Minimum trois captures d’écran : interface principale, fonctionnalité clé, affichage mobile.
- Petite démonstration de 2 à 5 minutes et checklist de tests signée par le mentor.

PROJET 05 — Portfolio personnel

Niveau	5 / Présentation professionnelle	Durée indicative	5 à 7 jours
Technologies	HTML + CSS + JavaScript	Déploiement	GitHub Pages (recommandé) ou Vercel

1. Objectif du projet

Créer une vitrine publique qui présente le développeur, son parcours et les projets réellement réalisés.

2. Utilisateur cible et contexte

Recruteur, enseignant, ami ou client potentiel.

3. Fonctionnalités obligatoires

- Créer les sections Accueil, À propos, Compétences, Projets et Contact.
- Afficher au minimum 3 projets déjà réalisés avec captures, description et lien GitHub.
- Ajouter une navigation utilisable sur ordinateur et mobile.
- Prévoir des liens visibles vers GitHub et une adresse e-mail.
- Fournir un bouton permettant d’accéder au CV PDF (facultatif si CV non prêt).
- Respecter la lisibilité des textes, les contrastes et les tailles d’écran.

4. Pages / écrans attendus

- Site mono-page à ancres ou plusieurs pages statiques.

5. Données et structure

Contenu statique provenant du code source ; formulaire serveur non exigé. Le contact peut être un lien mailto.

6. Règles métier et contraintes

- Ne pas inventer d’expériences ni de projets réalisés.
- Les images doivent avoir un texte alternatif pertinent.
- Le menu mobile doit être utilisable au clavier et au toucher.

7. Plan de réalisation conseillé

- Étape 1 : Dessiner une maquette.
- Étape 2 : Structurer les sections et données de projets.
- Étape 3 : Appliquer un design responsive.
- Étape 4 : Ajouter référencement élémentaire (title, description) et accessibilité.
- Étape 5 : Déployer et contrôler tous les liens.

8. Recette : tests d’acceptation

- Aucun lien mort ; au moins trois liens de dépôts fonctionnels.
- Le menu mobile s’ouvre et se ferme.
- Le site reste lisible sur écran étroit.

9. Bonus facultatifs

- Thème sombre.
- Version bilingue français/anglais.

10. Livrables et critères de validation

- Dépôt GitHub nommé clairement, avec commits significatifs et fichier README (objectif, installation, utilisation, captures, limites).
- Lien HTTPS du site réellement accessible ; aucune clé secrète publiée dans le dépôt.
- Minimum trois captures d’écran : interface principale, fonctionnalité clé, affichage mobile.
- Petite démonstration de 2 à 5 minutes et checklist de tests signée par le mentor.

PROJET 06 — Application météo

Niveau	6 / API et asynchrone	Durée indicative	5 à 8 jours
Technologies	HTML + CSS + JavaScript + Open-Meteo API	Déploiement	GitHub Pages ou Vercel

1. Objectif du projet

Récupérer puis représenter des données météo réelles provenant d'une API publique, sans clé secrète.

2. Utilisateur cible et contexte

Personne consultant les conditions météo de sa ville.

3. Fonctionnalités obligatoires

- Saisir un nom de ville et proposer un résultat géographique explicite.
- Récupérer latitude/longitude avec l'API de géocodage Open-Meteo.
- Afficher température actuelle, humidité, vent et état du ciel selon les champs disponibles.
- Présenter une prévision synthétique des prochains jours.
- Afficher chargement, ville introuvable, absence de connexion et erreur API.
- Afficher la date/heure et les unités des données ; attribuer la source Open-Meteo.

4. Pages / écrans attendus

- Recherche de ville ; résultat météo et cartes de prévision.

5. Données et structure

Données API non sensibles ; ville favorite éventuellement mémorisée en localStorage.

6. Règles métier et contraintes

- Ne pas inventer de météo en cas d'échec API.
- Ne pas confondre heure locale de la ville et heure du navigateur.
- Respecter les conditions d'utilisation non commerciales du fournisseur.

7. Plan de réalisation conseillé

- Étape 1 : Lire le format JSON de géocodage et prévision.
- Étape 2 : Créer recherche et requêtes fetch avec async/await.
- Étape 3 : Gérer les états de chargement et erreurs.
- Étape 4 : Afficher les données et icônes météo.
- Étape 5 : Déployer et vérifier les requêtes HTTPS.

8. Recette : tests d'acceptation

- Rechercher Kinshasa puis une autre ville.
- Une chaîne absurde produit un message « ville introuvable ».
- Couper Internet affiche un message utile plutôt qu'une page vide.

9. Bonus facultatifs

- Favoris.
- Graphique de température.

10. Livrables et critères de validation

- Dépôt GitHub nommé clairement, avec commits significatifs et fichier README (objectif, installation, utilisation, captures, limites).
- Lien HTTPS du site réellement accessible ; aucune clé secrète publiée dans le dépôt.
- Minimum trois captures d’écran : interface principale, fonctionnalité clé, affichage mobile.
- Petite démonstration de 2 à 5 minutes et checklist de tests signée par le mentor.

PROJET 07 — Gestion des notes des étudiants

Niveau	7 / Premières données persistantes	Durée indicative	1 à 2 semaines
Technologies	HTML + CSS + JavaScript + Supabase (Postgres)	Déploiement	Vercel (interface) + Supabase (base)

1. Objectif du projet

Permettre à un enseignant fictif d'enregistrer des étudiants, matières et notes, puis de calculer des moyennes fiables.

2. Utilisateur cible et contexte

Enseignant de démonstration ; aucun nom réel d'étudiant nécessaire.

3. Fonctionnalités obligatoires

- Ajouter, modifier et retirer un étudiant.
- Créer des matières avec coefficient entier positif.
- Saisir une note de 0 à 20 pour un étudiant et une matière.
- Afficher le relevé de notes et la moyenne pondérée.
- Lister les étudiants, rechercher par nom et trier par moyenne.
- Ajouter un compte enseignant via Supabase Auth et restreindre l'accès aux données.
- Afficher les erreurs de validation et les états de chargement.

4. Pages / écrans attendus

- Connexion ; Liste des étudiants ; Fiche étudiant ; Matières ; Saisie des notes.

5. Données et structure

Tables : students(id, owner_id, name, student_code), subjects(id, owner_id, name, coefficient), grades(id, student_id, subject_id, value). Relations et contrainte d'unicité étudiant/matière.

6. Règles métier et contraintes

- La moyenne = somme(note × coefficient) / somme(coefficients des matières notées).
- Une matière non notée ne doit pas être assimilée silencieusement à zéro.
- Activer Row Level Security (RLS) : chaque enseignant voit uniquement ses données.
- Ne jamais mettre de clé service_role Supabase dans le navigateur.

7. Plan de réalisation conseillé

- Étape 1 : Créer la maquette et le schéma SQL.
- Étape 2 : Configurer Supabase Auth, relations et politiques RLS.
- Étape 3 : Construire les formulaires CRUD et le calcul des moyennes.
- Étape 4 : Ajouter listes, recherche et états d'erreur.
- Étape 5 : Déployer sur Vercel et tester l'isolation entre deux comptes.

8. Recette : tests d'acceptation

- Notes 10 (coef. 1) et 16 (coef. 2) donnent moyenne 14.

- Notes < 0 ou > 20 refusées.
- Utilisateur B ne voit ni ne modifie les dossiers de A.

9. Bonus facultatifs

- Exporter un relevé imprimable en PDF depuis le navigateur.
- Ajouter groupes/classes.

10. Livrables et critères de validation

- Dépôt GitHub nommé clairement, avec commits significatifs et fichier README (objectif, installation, utilisation, captures, limites).
- Lien HTTPS du site réellement accessible ; aucune clé secrète publiée dans le dépôt.
- Minimum trois captures d’écran : interface principale, fonctionnalité clé, affichage mobile.
- Petite démonstration de 2 à 5 minutes et checklist de tests signée par le mentor.

PROJET 08 — Gestion des dépenses personnelles

Niveau	8 / CRUD + statistiques	Durée indicative	2 à 3 semaines
Technologies	React ou JavaScript + Supabase + Chart.js	Déploiement	Vercel + Supabase

1. Objectif du projet

Suivre un budget personnel avec comptes protégés et graphiques simples.

2. Utilisateur cible et contexte

Étudiant suivant son argent de poche et ses dépenses.

3. Fonctionnalités obligatoires

- Créer un compte et se connecter.
- Ajouter entrée ou sortie avec montant, date, catégorie et commentaire facultatif.
- Modifier et supprimer une transaction.
- Calculer recettes, dépenses et solde d’une période.
- Filtrer par mois et catégorie ; rechercher dans la liste.
- Afficher un graphique par catégorie et une évolution mensuelle.
- Afficher les montants dans la devise choisie par l’utilisateur (CDF ou USD), sans conversion automatique.

4. Pages / écrans attendus

- Connexion ; Tableau de bord ; Transactions ; Catégories/paramètres.

5. Données et structure

profiles(id, currency), categories(id, user_id, name, type), transactions(id, user_id, category_id, type, amount, date, note).

6. Règles métier et contraintes

- Montants strictement positifs ; un signe/type distinct détermine recette ou dépense.
- Ne pas additionner CDF et USD dans un même solde.
- Les données financières de démonstration sont fictives.
- Toutes les tables privées disposent de politiques RLS vérifiées.

7. Plan de réalisation conseillé

- Étape 1 : Écrire user stories et modèle relationnel.
- Étape 2 : Créer connexion et CRUD.
- Étape 3 : Implémenter calculs et filtres.
- Étape 4 : Ajouter graphiques et adaptation mobile.
- Étape 5 : Déployer puis tester deux comptes différents.

8. Recette : tests d’acceptation

- Recettes 100, dépenses 30 et 20 donnent solde 50 dans une même devise.

- Un filtre mensuel n’affiche pas les autres mois.
- Aucun utilisateur ne peut accéder aux transactions d’un autre.

9. Bonus facultatifs

- Objectifs de budget mensuel.
- Export CSV.

10. Livrables et critères de validation

- Dépôt GitHub nommé clairement, avec commits significatifs et fichier README (objectif, installation, utilisation, captures, limites).
- Lien HTTPS du site réellement accessible ; aucune clé secrète publiée dans le dépôt.
- Minimum trois captures d’écran : interface principale, fonctionnalité clé, affichage mobile.
- Petite démonstration de 2 à 5 minutes et checklist de tests signée par le mentor.

PROJET 09 — Mini-boutique e-commerce (simulation)

Niveau	9 / Workflow complet	Durée indicative	3 à 4 semaines
Technologies	React + Supabase Auth/Postgres	Déploiement	Vercel + Supabase

1. Objectif du projet

Simuler une boutique complète sans vraie collecte d’argent ni intégration d’un paiement payant.

2. Utilisateur cible et contexte

Visiteur acheteur fictif et administrateur de démonstration.

3. Fonctionnalités obligatoires

- Afficher produits avec photo, prix, stock et catégorie.
- Rechercher et filtrer les produits.
- Ajouter au panier, changer les quantités et retirer un article.
- Créer un compte client et valider une commande fictive.
- Afficher historique et détail des commandes du client.
- Proposer un écran administrateur pour ajouter/modifier les produits et consulter les commandes.
- Réduire le stock correctement au moment de la commande.

4. Pages / écrans attendus

- Catalogue ; Fiche produit ; Panier ; Connexion ; Mes commandes ; Administration.

5. Données et structure

products(id, name, price, stock, category), orders(id, user_id, status, total), order_items(id, order_id, product_id, quantity, unit_price), profiles(id, role).

6. Règles métier et contraintes

- Aucun vrai paiement ni donnée bancaire à recueillir.
- Un client ne peut pas s’attribuer le rôle admin depuis le navigateur.
- Le montant d’une commande se recalcule côté service de confiance ; ne pas accepter le total déclaré par le client.
- Stock et commande doivent être mis à jour de manière atomique (fonction SQL/RPC sécurisée) pour éviter le surstock.
- Créer une politique RLS sur chaque table exposée.

7. Plan de réalisation conseillé

- Étape 1 : Concevoir catalogue et panier local.
- Étape 2 : Modéliser tables et règles d’accès.
- Étape 3 : Ajouter Auth et commandes simulées.
- Étape 4 : Développer fonctions sécurisées de commande et interface admin.
- Étape 5 : Tester les stocks, les rôles et déployer.

8. Recette : tests d’acceptation

- Produit de stock 2 : commande de 3 unités refusée.
- Le client A ne peut pas voir la commande de B.
- Un utilisateur classique ne peut ni modifier un prix ni augmenter son rôle.

9. Bonus facultatifs

- Codes promotionnels fictifs.
- Images produit et pagination.

10. Livrables et critères de validation

- Dépôt GitHub nommé clairement, avec commits significatifs et fichier README (objectif, installation, utilisation, captures, limites).
- Lien HTTPS du site réellement accessible ; aucune clé secrète publiée dans le dépôt.
- Minimum trois captures d’écran : interface principale, fonctionnalité clé, affichage mobile.
- Petite démonstration de 2 à 5 minutes et checklist de tests signée par le mentor.

PROJET 10 — Plateforme de gestion académique

Niveau	10 / Projet de synthèse	Durée indicative	4 à 6 semaines
Technologies	React + TypeScript + Supabase (Auth + Postgres)	Déploiement	Vercel + Supabase

1. Objectif du projet

Créer un prototype académique multi-rôles utilisable dans un environnement de démonstration.

2. Utilisateur cible et contexte

Administrateur, enseignant et étudiant (comptes de test).

3. Fonctionnalités obligatoires

- Authentifier les utilisateurs avec des rôles distincts : admin, enseignant et étudiant.
- Administrateur : créer classes, matières, périodes et inscriptions.
- Enseignant : visualiser ses classes et encoder/modifier les notes qui lui sont attribuées.
- Étudiant : consulter uniquement ses matières, notes et moyennes publiées.
- Calculer des moyennes pondérées et afficher une synthèse par période.
- Générer un bulletin imprimable depuis le navigateur.
- Afficher un tableau de bord adapté à chaque rôle.
- Tracer les changements importants (auteur, date, ancienne/nouvelle valeur si nécessaire).

4. Pages / écrans attendus

- Connexion ; Tableau de bord par rôle ; Étudiants ; Classes ; Matières ; Inscriptions ; Notes ; Bulletins ; Audit.

5. Données et structure

profiles, classes, subjects, periods, enrollments, teaching_assignments, grades, audit_logs ; clés étrangères, index et contraintes d’unicité.

6. Règles métier et contraintes

- Toute permission métier doit être appliquée en base par RLS et/ou fonctions sécurisées, pas seulement par masquage de boutons.
- Une note doit être comprise entre 0 et 20 ; pas de doublon étudiant-matière-période sans politique explicite.
- L’étudiant ne voit que ses propres notes publiées, pas les brouillons.
- Utiliser des profils de test fictifs ; ne pas collecter de données scolaires réelles.
- Les opérations sensibles doivent être auditables ; les modifications de rôle sont réservées aux administrateurs via mécanisme sécurisé.

7. Plan de réalisation conseillé

- Étape 1 : Rédiger les cas d’utilisation et rôles.
- Étape 2 : Dessiner modèle relationnel et règles RLS.
- Étape 3 : Construire inscription, classes, matières et affectations.
- Étape 4 : Implémenter saisie, publication et consultation des notes.

- Étape 5 : Ajouter bulletins, tableaux de bord, journal et tests.
- Étape 6 : Déployer, préparer comptes de démonstration et documenter les limites.

8. Recette : tests d’acceptation

- L’étudiant A ne lit pas les notes de B.
- Un enseignant non affecté ne peut pas éditer une note.
- Une note brouillon n’apparaît pas chez l’étudiant.
- Une moyenne avec coefficients est exacte et reproductible.

9. Bonus facultatifs

- Import CSV d’étudiants fictifs.
- Statistiques de réussite par classe.

10. Livrables et critères de validation

- Dépôt GitHub nommé clairement, avec commits significatifs et fichier README (objectif, installation, utilisation, captures, limites).
- Lien HTTPS du site réellement accessible ; aucune clé secrète publiée dans le dépôt.
- Minimum trois captures d’écran : interface principale, fonctionnalité clé, affichage mobile.
- Petite démonstration de 2 à 5 minutes et checklist de tests signée par le mentor.

ANNEXE — MISE EN LIGNE GRATUITE

Route A : GitHub Pages, projets statiques

- Créer un dépôt GitHub contenant au minimum index.html, style.css et script.js (les noms peuvent varier).
- Ouvrir les paramètres du dépôt, rubrique Pages ; sélectionner la branche et le dossier de publication.
- Ouvrir l'URL publiée, vérifier les chemins relatifs, les images et le comportement mobile.
- Documenter l'adresse du site dans le README.

Route B : Vercel pour interfaces web

- Pousser le projet sur GitHub puis importer le dépôt dans Vercel.
- Pour un site HTML simple, sélectionner les paramètres de build adaptés ; pour React/Vite, vérifier la commande de build et le dossier dist.
- Lancer le déploiement et vérifier l'URL publique en HTTPS.
- Utiliser uniquement des valeurs publiques dans les variables front-end ; les secrets ne doivent jamais y être exposés.

Route C : Supabase pour les données

- Créer un projet gratuit (dans la limite du nombre de projets actifs permis).
- Créer les tables, clés étrangères et contraintes depuis l'éditeur SQL.
- Configurer l'authentification et les URL de redirection autorisées.
- Activer RLS et écrire des politiques adaptées ; tester avec deux comptes aux droits différents.
- Utiliser la clé publique prévue pour le client ; ne jamais exposer la service_role key.
- Relier le client hébergé sur Vercel au projet Supabase puis tester le parcours complet.

Route D : Firebase (alternative)

- Créer un projet Firebase au plan Spark lorsque les services utilisés le permettent.
- Pour une application statique, activer Firebase Hosting ; pour les données, Firestore et ses règles de sécurité.
- Contrôler les quotas et les conditions d'accès ; certaines fonctionnalités cloud nécessitent de passer à un plan facturé.

Ressources et conditions officielles à vérifier avant lancement

https://docs.github.com/en/pages

https://vercel.com/docs/plans/hobby

https://supabase.com/pricing

https://supabase.com/docs/guides/platform/free-project-pausing

https://firebase.google.com/docs/projects/billing/firebase-pricing-plans

https://open-meteo.com/en/docs

Fiche de suivi à reproduire pour chaque projet

- Date de début : _____ ; date de fin : _____ ; note obtenue : ____ / 20.
- Dépôt GitHub : _____ ; lien de démo : _____.
- Tests obligatoires réussis : ____ / ____ ; défauts restant à corriger : _____.
- Décision du mentor : validé / à corriger ; prochaine compétence à travailler : _____.